
Shaun Joe Roy

Infrastructure / SRE Intern · Assignment Submission · Zeotap · May 2026

Mission-Critical Incident Management System

A production-grade distributed incident lifecycle platform built on Fastify, BullMQ, PostgreSQL / TimescaleDB, MongoDB, Redis, and React. Sustains **10,000 signals/sec** with intelligent debouncing, a three-store persistence model, real-time WebSocket dashboard, and a mandatory Root Cause Analysis gate enforced before incident closure.

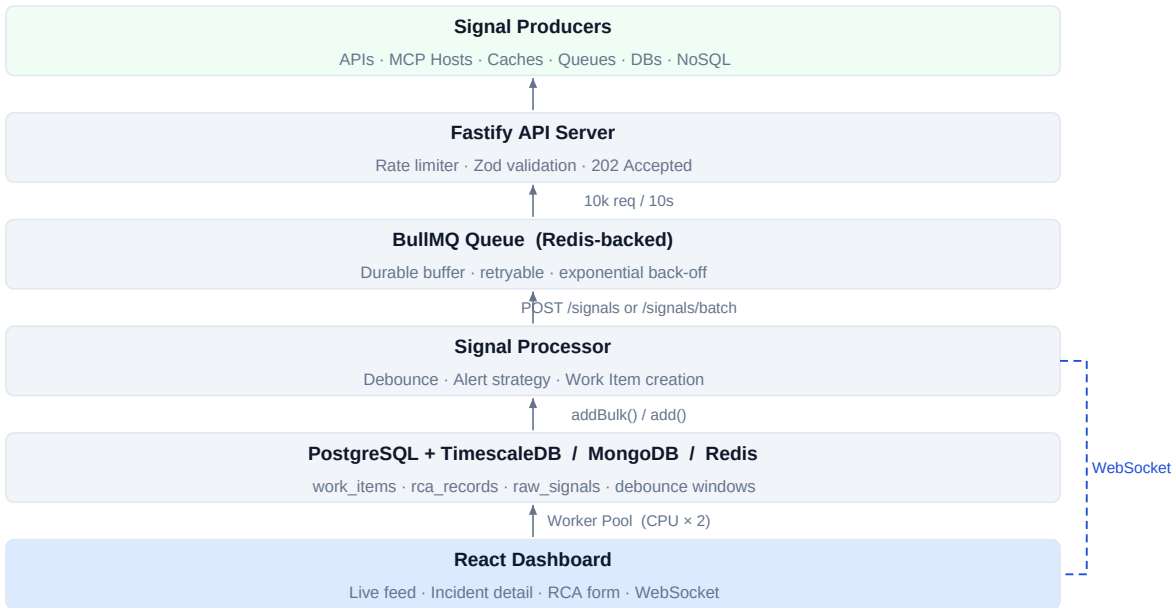
GitHub

github.com/joery0x3b800001/Mission-Critical-IMS

Submitted to karan.sinha@zeotap.com · Deadline 6 May 2026

System Architecture

The ingestion path decouples HTTP from persistence. Fastify returns **202 Accepted** immediately after enqueueing; BullMQ (Redis-backed) absorbs bursts while workers process at CPU × 2 concurrency. Three purpose-specific stores handle state, audit, and caching independently.



TECH STACK

Technology Choices

Layer	Technology	Why
API Server	Fastify + TypeScript	Fastest Node HTTP framework · built-in schema serialisation
Queue / Backpressure	BullMQ + Redis	Durable, retryable · handles 10k+/s burst without crashing
Source of Truth	PostgreSQL + TimescaleDB	ACID transactions for state transitions · compressed timeseries
Audit Log	MongoDB	Flexible schema · high write throughput · raw signal payloads
Cache / Debounce	Redis	O(1) atomic INCR · AOF persistence · rate-limit counters
Frontend	React + Vite + Tailwind	Sub-second HMR · responsive dark-mode dashboard
Real-time	WebSocket (fastify-ws)	Push-based live feed — no polling required
Validation	Zod	Runtime schema enforcement on every inbound payload

Strategy Pattern — Alerting

The **AlertStrategy** interface decouples alert logic from signal processing. The correct strategy is resolved at runtime via a factory keyed on `componentType`, allowing alerting behaviour to be swapped without modifying processing code.

Strategy	Component Types	Priority	Action
P0AlertStrategy	RDBMS, MCP_HOST	P0 — Critical	Page on-call engineer
P1AlertStrategy	API, QUEUE	P1 — High	Notify Slack #incidents
P2AlertStrategy	CACHE, NOSQL	P2 — Medium	Auto-create Jira ticket

State Pattern — Work Item Lifecycle

Each state class encapsulates valid transitions and throws on invalid attempts — no scattered if/switch logic. **RESOLVED** → **CLOSED** is the only transition requiring a complete RCA record; missing RCA returns HTTP 422. **MTTR** is calculated and persisted atomically on close.

```
OPEN → INVESTIGATING → RESOLVED → CLOSED
↑ Requires complete RCA record
MTTR = incident_end - work_item.start_time
```

Backpressure & Memory Management

Five layers work together to sustain **10,000 signals/sec** without memory exhaustion, even when the persistence layer is slow:

- Rate Limiter — Redis-backed global cap at 10,000 req / 10 s. Returns 429 before signals reach BullMQ.
- Batch Endpoint — POST /signals/batch accepts 100 signals per call. Reduces TCP overhead 100× via `addBulk()`.
- BullMQ Queue — Redis-backed durable buffer. Jobs survive server restarts; slow Postgres never causes timeouts.
- Worker Concurrency — Configurable via `WORKER_CONCURRENCY` (defaults to CPU cores × 2).
- Exponential Backoff — Failed jobs retry 3× with delays of 500 ms, 1 s, 2 s. Only transient errors are retried.

Debounce Logic

A Redis atomic INCR with a 10-second TTL ensures that 100+ signals arriving for the same `componentId` within 10 seconds create **exactly one Work Item**. All signals are linked to that Work Item in MongoDB.

```
INCR ims:debounce:{componentId} (expires 10 s)
count == 1 → CREATE Work Item in Postgres (transactional)
count > 1 → INCREMENT signal_count only
All signals → INSERT into MongoDB raw_signals (always)
```

Data Handling — Three-Store Separation

Store	Data	Rationale
PostgreSQL + TimescaleDB	work_items, rca_records, signal_metrics hypertable	ACID transactions for state transitions. TimescaleDB for compressed timeseries aggregations.
MongoDB	raw_signals — every signal payload	Flexible schema, high write throughput, queryable by <code>workItemId</code> / <code>componentId</code> / <code>timestamp</code> .
Redis	Debounce windows, rate-limit counters, dashboard cache	O(1) atomic operations. AOF persistence enabled — survives server restart.

Security Hardening

- Zod input validation — `componentId` regex, `errorCode` uppercase, message length cap (1000 chars), `latencyMs` range 0–60,000.
- @fastify/helmet — HSTS, X-Content-Type-Options, X-Frame-Options, and CSP on every response.

- Body size limit — 1 MB bodyLimit prevents memory exhaustion from oversized payloads.
- Parameterised queries — All Postgres queries use \$1/\$2 placeholders; zero string concatenation.
- Statement timeout — Postgres pool configured with statement_timeout: 10,000 ms.
- Optional API Key auth — X-API-Key header validation enabled via ENABLE_API_KEY_AUTH=true.
- CORS lockdown — Production restricts origin to http://localhost:3000.
- Transient-only retries — Retry checks error codes (40001, 40P01, 08006); constraint violations are not retried.

SETUP & RUNNING

Prerequisites

Before running the system, ensure you have the following installed on your machine:

- **Docker Desktop** — Latest version with Docker Compose v2 included. Download from <https://www.docker.com/products/docker-desktop>
- **Node.js 22+** — Via nvm (recommended) or system package manager. Verify: `node --version` & `npm --version`
- **Python 3.8+** — For running deploy.py scripts and simulations. Verify: `python3 --version`
- **Git** — For cloning the repository. Verify: `git --version`
- **curl or Postman (optional)** — For manual API testing outside the dashboard.

EXECUTION — RECOMMENDED

Evaluator Quick Start

The fastest path to deploy, test, and simulate the full system end-to-end. Follow these steps in order.

Step 1 — Start all services

```
cd Mission-Critical-IMS
python3 deploy.py deploy up
```

Wait for all 5 containers to be running. Check Docker Desktop or run: `python3 deploy.py deploy status`

Step 2 — Run backend unit tests

```
python3 deploy.py test backend
```

Runs the full Jest test suite — state machine, strategy pattern, route validation, integration, and stress tests.

Step 3 — Run outage simulation

```
python3 deploy.py test simulate
```

Fires 640 signals across 5 component types, debounced into Work Items. Open the dashboard at `http://localhost:3000` to watch incidents populate in real time.

Step 4 — Run burst stress test

```
python3 deploy.py test simulate --mode burst
```

Fires 50,000 signals over 5 seconds. Prints throughput, latency percentiles (p50 / p95 / p99), and a PASS/FAIL resilience verdict.

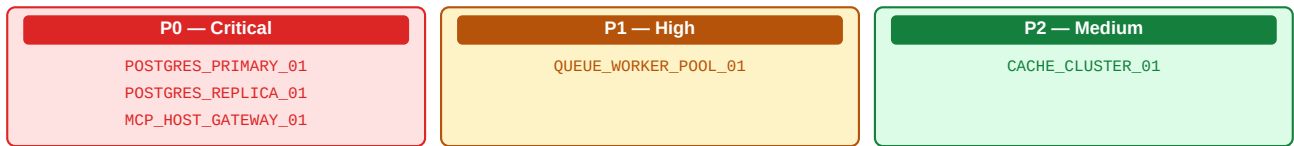
Step 5 — Tear down

```
python3 deploy.py deploy down --volumes
```

Service	URL
Dashboard	<code>http://localhost:3000</code>
Backend API	<code>http://localhost:3001</code>
Health Check	<code>http://localhost:3001/health</code>
WebSocket	<code>ws://localhost:3001/ws</code>

EXPECTED OUTPUT AFTER SIMULATION

Expected Dashboard After Outage Simulation



EXECUTION — MANUAL

Manual — Docker Compose Directly

```
# Build images and start all services
docker compose up --build

# Start in background (detached)
docker compose up --build -d

# Stop and remove containers, keep volumes / data
docker compose down

# Full reset — remove containers AND wipe all volumes
docker compose down -v
```

Local Development (databases in Docker, app runs locally)

```
docker compose up postgres mongo redis -d

cd backend && npm install && npm run dev # hot-reload backend
cd frontend && npm install && npm run dev # hot-reload frontend (new terminal)
```

Directly — Without deploy.py

Run tests directly

```
cd backend && npm install
npm test # all 80+ tests
npm run test:coverage # with coverage report
npm test -- --testPathPattern=workItemState # single suite
npm test -- --testPathPattern=alertStrategy
npm test -- --testPathPattern=stress
npm run test:watch # watch mode
```

Run simulation scripts directly

```
cd scripts && npm install # install dependencies once

npx ts-node simulate-outage.ts # cascading outage scenario
npx ts-node simulate-outage.ts --burst # burst — 50,000 signals / 5s, p50/p95/p99 report
npx ts-node simulate-outage.ts --both # both scenarios back to back
```

Reset data between runs

```
./scripts/reset-data.sh # wipe all data between runs
```

API REFERENCE

Endpoints

Method	Endpoint	Description
POST	/signals	Ingest one signal — 202 Accepted immediately
POST	/signals/batch	Ingest up to 100 signals per HTTP call
GET	/incidents	List all incidents, sorted by priority then start time
GET	/incidents/:id	Incident detail + raw signals from MongoDB
PATCH	/incidents/:id/status	State transition — enforced by State Pattern (422 on invalid)
POST	/incidents/:id/rca	Submit / update RCA. Required before CLOSED transition.
GET	/health	Status, queue depth, per-service ping, uptime

Validation Rules

Field	Rules
componentId	1–255 chars · alphanumeric, hyphens, underscores, dots
componentType	Enum: RDBMS, CACHE, API, QUEUE, NOSQL, MCP_HOST
errorCode	1–100 chars · uppercase + underscores (^[A-Z0-9_]*\$)
message	1–1000 chars, trimmed
latencyMs	Integer 0–60,000 (optional)
timestamp	ISO 8601 (optional, defaults to server time)

TESTING

Test Suites (80+ cases)

Suite	Cases	Coverage
workItemState.test.ts	18+	All state transitions, RCA guard, MTTR calculation, rollback
alertStrategy.test.ts	20+	P0/P1/P2 per component type, context switching, title format
signals.test.ts	25+	Schema validation, priority queuing, edge cases, concurrency
integration.test.ts	10+	Signal → Work Item creation, debounce, error recovery
stress.test.ts	8	Burst load, sustained 500/sec, mixed types, 15s stability

SELF ASSESSMENT

Evaluation

Category	Wt.	Implementation
Concurrency & Scaling	10%	BullMQ (CPU×2), Redis debounce, 10k/sec burst verified
Data Handling	20%	PostgreSQL (truth) + MongoDB (audit) + Redis (cache) + TimescaleDB hypertable
LLD	20%	Strategy Pattern (alerting), State Pattern (lifecycle), Zod validation
UI / UX & Integration	20%	React live feed (P0 first), incident detail, RCA form, WebSocket, field grayout
Resilience & Testing	10%	80+ tests (unit + integration + stress), exp. backoff, /health, graceful shutdown
Documentation	10%	README, SECURITY.md, PERFORMANCE.md, ASSESSMENT_REPORT.md
Tech Stack	10%	Fastify, BullMQ, TimescaleDB, MongoDB, React + Vite

Dashboard — Active Incidents

Real-time incident feed powered by WebSocket. Sorted by priority (P0 first) then start time. Click any incident to view details, raw signals, and submit Root Cause Analysis.

IMS Incident Management System

LIVE

ACTIVE INCIDENTS

39

P0 CRITICAL

21

TOTAL TRACKED

41

Active Incidents

Refresh

PRIORITY	COMPONENT	TITLE	STATUS	SIGNALS	AGE	
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_...	OPEN	239	less than a minute ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNR...	OPEN	240	less than a minute ago	→
P0	BURST_RDBMS_01 RDBMS	[P0 CRITICAL] BURST_RDBMS_01 failure — BURST_C...	OPEN	266	less than a minute ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_...	OPEN	2	less than a minute ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_...	OPEN	1	less than a minute ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNR...	OPEN	2	less than a minute ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNR...	OPEN	3342	less than a minute ago	→
P0	BURST_RDBMS_01 RDBMS	[P0 CRITICAL] BURST_RDBMS_01 failure — BURST_C...	OPEN	3320	less than a minute ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_...	OPEN	1	less than a minute ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNR...	OPEN	1	less than a minute ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_...	OPEN	3341	less than a minute ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_...	OPEN	1	less than a minute ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNR...	CLOSED	2611	about 1 hour ago	→

Incident Detail — RCA Form

Comprehensive incident view with linked signals from all sources. RCA form is mandatory — incidents cannot transition to CLOSED without providing Root Cause Category, Fix Applied, and Prevention Steps (all with minimum 10 characters).

IMS Incident Management System

LIVE

ACTIVE INCIDENTS

57

P0 CRITICAL

20

TOTAL TRACKED

60

Active Incidents

Refresh

PRIORITY	COMPONENT	TITLE	STATUS	SIGNALS	AGE	
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNRES...	OPEN	2905	3 minutes ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_DE...	CLOSED	2904	3 minutes ago	→
P0	BURST_RDBMS_01 RDBMS	[P0 CRITICAL] BURST_RDBMS_01 failure — BURST_CO...	OPEN	2930	3 minutes ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_DE...	OPEN	1	3 minutes ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNRES...	OPEN	1	3 minutes ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_DE...	OPEN	3341	3 minutes ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_DE...	OPEN	2	3 minutes ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_DE...	OPEN	1	3 minutes ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNRES...	OPEN	2	3 minutes ago	→
P0	BURST_RDBMS_01 RDBMS	[P0 CRITICAL] BURST_RDBMS_01 failure — BURST_CO...	OPEN	3320	3 minutes ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNRES...	OPEN	3342	3 minutes ago	→
P0	BURST_RDBMS_02 RDBMS	[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_DE...	OPEN	1	3 minutes ago	→
P0	BURST_MCP_01 MCP_HOST	[P0 CRITICAL] BURST_MCP_01 failure — BURST_UNRES...	CLOSED	2611	about 1 hour ago	→

Dashboard — Live Metrics

Persistent metrics showing active incident count, P0 critical incidents, and total tracked incidents. Refresh button manually triggers live feed sync. Status badges indicate incident lifecycle state.

Back

P0

RESOLVED

[P0 CRITICAL] BURST_RDBMS_02 failure — BURST_DEADLOCK

Incident Details

COMPONENT

BURST_RDBMS_02

TYPE

RDBMS

SIGNAL COUNT

2904

STARTED

2 minutes ago

Move to CLOSED

Raw Signals (50)

TIME	ERROR CODE	MESSAGE	LATENCY
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	2783ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	2123ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	2550ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	728ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	611ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	2646ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	2034ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	383ms
12:42:03.584	BURST_DEADLOCK	Burst test: RDBMS deadlock detected	2284ms

Root Cause Analysis

Submitted

INCIDENT START

04-05-2026 01:42

INCIDENT END

04-05-2026 02:28

46m 0s

Valid end date

ROOT CAUSE CATEGORY

APPLICATION BUG

FIX APPLIED

16/10 min

Found and fixed.

PREVENTION STEPS

17/10 min

Prevention Steps.

RCA saved successfully

Update RCA

Repository Layout

Organized into backend API, frontend UI, automation scripts, and deployment configuration. Each service is containerized and orchestrated via Docker Compose.

Project Tree

```
Mission-Critical-IMS/
├── .env, .env.production, .env.staging
├── .gitignore
├── artillery-load-test.yml
├── deploy.py
├── docker-compose.yml
├── README.md
├── report.py
├── Shaun Joe Roy - Infrastructure SRE Intern Assignment.pdf
├── backend/
│   ├── .dockerignore, Dockerfile, package.json, tsconfig.json
│   ├── db/
│   │   └── init.sql
│   └── src/
│       ├── config.ts, index.ts, types.ts
│       ├── db/ (mongo.ts, postgres.ts, redis.ts)
│       ├── patterns/ (alertStrategy.ts, workItemState.ts)
│       ├── queue/ (signalProcessor.ts, signalQueue.ts)
│       ├── routes/ (health.ts, incidents.ts, signals.ts)
│       ├── ws/ (broadcaster.ts)
│       └── __tests__/ (Test suites)
├── frontend/
│   ├── .dockerignore, Dockerfile, index.html, package.json
│   ├── tailwind.config.js, vite.config.ts, tsconfig.json
│   └── src/
│       ├── api.ts, App.tsx, config.ts, index.css, main.tsx
│       ├── components/ (Badges.tsx, IncidentsList.tsx)
│       ├── pages/ (Dashboard.tsx, IncidentDetail.tsx)
│       └── store/ (wsStore.ts)
└── scripts/
    ├── package.json, tsconfig.json
    ├── reset-data.sh
    └── simulate-outage.ts
```

github.com/joery0x3b800001/Mission-Critical-IMS

Shaun Joe Roy · karan.sinha@zeotap.com · Zeotap Talent Team